

```
In [1]: import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
```

```
In [2]: df=pd.read_csv("train.csv")
```

```
In [3]: df.head()
```

```
Out[3]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05

```
In [4]: print(df.isnull().sum())
df = df[['Pclass', 'Sex', 'Embarked', 'Survived']]
df = df.dropna()
```

```
PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age            177
SibSp           0
Parch           0
Ticket          0
Fare            0
Cabin          687
Embarked        2
dtype: int64
```

```
In [5]: print(df.isnull().sum())
```

```
Pclass      0
Sex          0
Embarked     0
Survived     0
dtype: int64
```

```
In [6]: df['Sex']=df['Sex'].map({'male':0, 'female':1})
df['Embarked']=df['Embarked'].map({'S':0, 'C':1, 'Q':2})
```

```
In [7]: X=df[['Pclass', 'Sex', 'Embarked']]
y=df['Survived']
```

```
In [8]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
```

```
In [9]: from sklearn.tree import DecisionTreeClassifier
```

```
In [10]: dt = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt.fit(X_train, y_train)
```

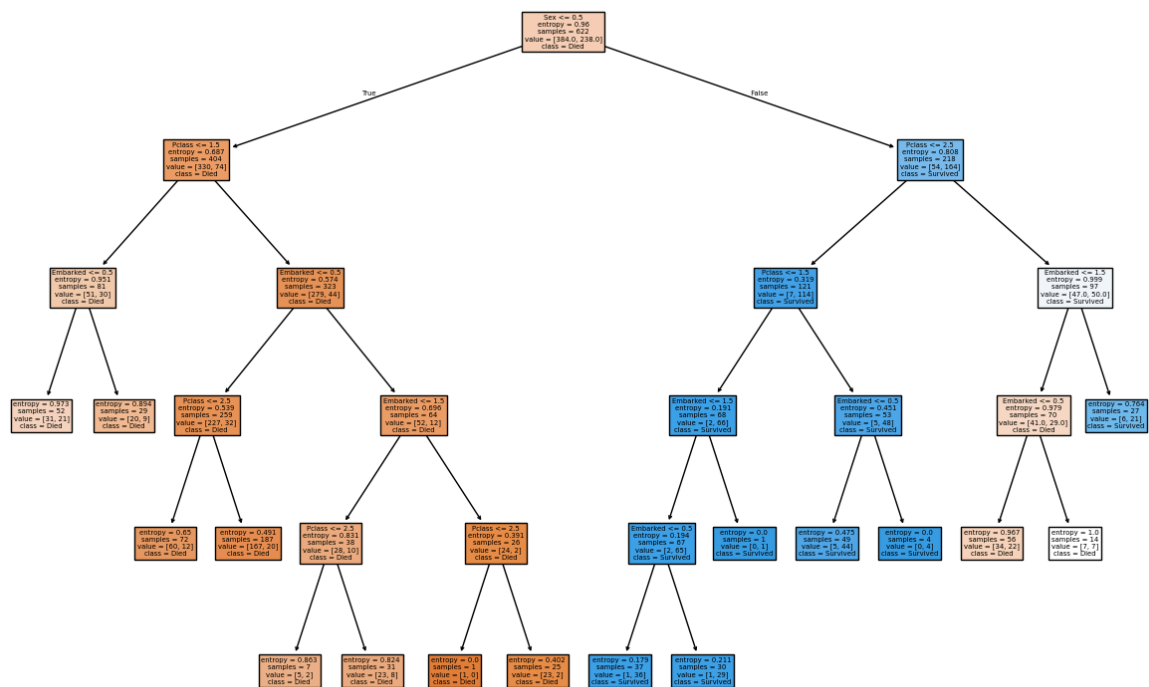
```
Out[10]: ▾ DecisionTreeClassifier ⓘ ?
          ► Parameters
```

```
In [11]: y_pred = dt.predict(X_test)
```

```
In [12]: from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

```
In [13]: plt.figure(figsize=(15, 10))
plot_tree(dt,
          feature_names=['Pclass', 'Sex', 'Embarked'],
          class_names=['Died', 'Survived'],
          filled=True)

plt.show()
```



```
In [14]: probs = y_train.value_counts(normalize=True)
```

Trying manual calculation to perform entropy

```
In [15]: df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

```
def find_entropy(y):
    values, counts = np.unique(y, return_counts=True)
    entropy = 0
    total = counts.sum()
    for c in counts:
        p = c / total
        entropy -= p * np.log2(p)
    return entropy
```

```
In [ ]: def find_information_gain(X, y, feature):
    total_entropy = find_entropy(y)
    values, counts = np.unique(X[feature], return_counts=True)
    weighted_entropy = 0

    for v, c in zip(values, counts):
        subset_y = y[X[feature] == v]
        subset_entropy = find_entropy(subset_y)
        weighted_entropy += (c / len(y)) * subset_entropy

    information_gain = total_entropy - weighted_entropy
    return information_gain

root_entropy = find_entropy(y)
print(f"Total Entropy (Survived): {root_entropy:.4f}")
```

Total Entropy (Survived): 0.9598

```
In [17]: features = ["Pclass", "Sex", "Embarked"]
         ig_results = {}

         for feat in features:
             ig = find_information_gain(X, y, feat)
             ig_results[feat] = ig
             print(f"Information Gain for {feat}: {ig:.4f}")
```

Information Gain for Pclass: 0.0824

Information Gain for Sex: 0.2160

Information Gain for Embarked: 0.0210

```
In [18]: root_node = max(ig_results, key=ig_results.get)
         print(f"\nRoot Node selected by ID3 manual calculation: {root_node}")
```

Root Node selected by ID3 manual calculation: Sex